MY LITTLE RPG - ÉTAPE 4 INTÉGRATION ET DÉPLACEMENT DU PERSONNAGE



HENRI PITRE - 2432978

RECHERCHE

Répondez à ces questions pour démarrer votre réflexion sur les jetons de sécurité. Le pointage de cette section est noté pour l'équipe, et pas de façon individuelle.

Dans nos sites habituels (PHP, ASP.NET), comment est-ce qu'on gère l'inscription, l'authentification et l'autorisation?

En s'inscrivant, on crée les crédentiels de l'utilisateur et on les enregistre sans communiquer publiquement celles sensibles. Également, on associe le mot de passe à un hash pour une sécurité sur les accès en texte clairs.

Lors de la connexion, il communique au serveur pour vérifier que les identifiants sont valides en les comparant, sans que la connexion donne la réponse de ce qui était attendu.

Lorsque que la connexion est établie, on crée un cookie contenant les identifiants non-sensibles de la session (moins de données, mieux c'est) et pour chaque page reliée qu'il charge, il montre le cookie de connexion au préalable pour leur informer des données de session.

Si l'utilisateur est authentifié et autorisé (si non public), il peut avoir accès à la ressource/page sécuritairement.

De quelle façon ces opérations diffèrent-elles pour un site utilisant le paradigme Frontend/Backend?

Puisqu'on divise en deux le site avec une telle méthode, on utilise plutôt un jeton de connexion qui est communiqué à la demande valide du client par l'API serveur. Ensuite, à chaque fois qu'une ressource nécesite une connexion/autorisation, l'API se fie au jeton qui est associé à la connexion client avant de donner l'accès.

©2025 RTL, PAS

Pour une API où on a aucune authentification, quels seraient les principaux vecteurs d'attaque ou d'abus?

L'impact direct serait sur les données communiquées. À l'aide des routes (endpoints), qui sont complètement exposées, l'attaque serait centralisée sur la modification, la suppression ou même l'ajout de données. Dans le cas d'une API vraiment mal sécurisée et conceptualisée, un utilisateur pourrait même obtenir et modifier les informations destinées à l'Admin en utilisant les routes Admin créées au préalable.

Pourquoi est-il une mauvaise idée d'envoyer les informations de connexion à chaque demande faite auprès de l'API?

Pour une raison principale, la sécurité. En effet, cela pose un grand pronblème car si on répète les demandes, on peut alors plus facilement repérer les demandes d'authentification lors du voyage client → serveur. Ainsi, quelqu'un pourrait l'intercepter et tenter de déchiffrer le message de communication. De plus, si une personne arrivait à déchiffrer cette information, il tomberait directement sur les identifiants utilisateurs, au lieu de tomber sur un token d'autorisation qui a une durée de vie. Également, cela ralentirait le serveur en créant des requêtes excessives.

Regardez sur le site relié ICI pour voir comment un jeton JWT est encodé. Quelles informations sont véhiculées dans le jeton?

Le jeton est divisé en trois sections :
Le header, qui spécifie le but et le type d'encodage du jeton;
Le Payload, qui donne les détails public du jeton tels que son nom, son créateur, etc.;
Sa signature, qui permet de s'assurer que le jeton n'est pas une contrefaçon.

Mettez-vous du point de vue d'un acteur malveillant. Vous avez réussi à intercepter le jeton d'accès d'un utilisateur. Quelles opérations pouvez-vous accomplir en son nom? Comment les systèmes de jeton de sécurité résolvent-ils ce problème?

Le jeton permettant l'accès au compte de l'utilisateur est sécurisé de plusieurs façon. Dans l'API MyLittleRPG, il est par exemple vivant pendant seulement quelques minutes, ce qui empêche des changements majeurs ou une intrusion discrète continue. Cependant, il permet tout de même la connexion au système et l'accès. En tant que tel, un jeton est utile une seule fois, et ne peut être modifié sans que la signature soit intacte, alors il ne peut rien faire de plus, excepté dans

©2025 RTL, PAS

À la lumière de ce que vous comprenez des jetons de sécurité, comment pensez-vous qu'un service de connexion comme « Login with Facebook » permet-il de fournir de façon sécuritaire une méthode de connexion universelle à des sites variés, qui n'appartiennent peut-être même pas à l'écosystème Facebook?

Selon ce que j'ai compris et experimenté avec ces jetons, la connexion est très sécurisée car elle est gérée en externe. En effet, le site qui permet la connexion avec Facebook fait simplement déléguer le travail à celui-ci. En indiquant à Facebook qu'une personne doit se connecter, Facebook gère le reste selon les informations demandées du site (Permission d'accès au nom d'utilisateur, etc.) Et n'en fait aucunement part au site. Ainsi, la source ne voit ni s'occupe d'absolument rien de ce qui se passe pendant que l'utilisateur utilise ce moyen de connexion. Lorsque c'est terminé, Facebook l'indique et envoie sa confirmation (qui contient un jeton, je suppose), qui n'indique pas le mot de passe Facebook au site primaire.

©2025 RTL, PAS

LIVRABLES

Fonctionnalités à livrer :

Dans l'API :

- Modèles pour le personnage et les utilisateurs
- Endpoints fonctionnels pour les comptes, l'accès aux informations du personnage et pour le déplacement.
- Logique de mouvement avec validation
- Logique d'exploration

Dans l'application Web :

- Interface d'inscription et connexion
- Chargement et affichage des informations du personnage
- Déplacement du personnage dans le monde
- Exploration de tuiles visibles en cliquant dessus
- Changement de l'affichage pour refléter le déplacement (tuiles, informations de position, etc.)

Éléments à livrer

- Courte recherche sur les jetons de sécurité incluant les réponses aux questions de la section à ce sujet.
- Lien GitHub vers le projet API.
- (Au besoin si c'est un Repo différent) Lien GitHub vers le projet de site web.

©2025 RTL, PAS